

Trees and Binary Tree

11 May 2026 19:58

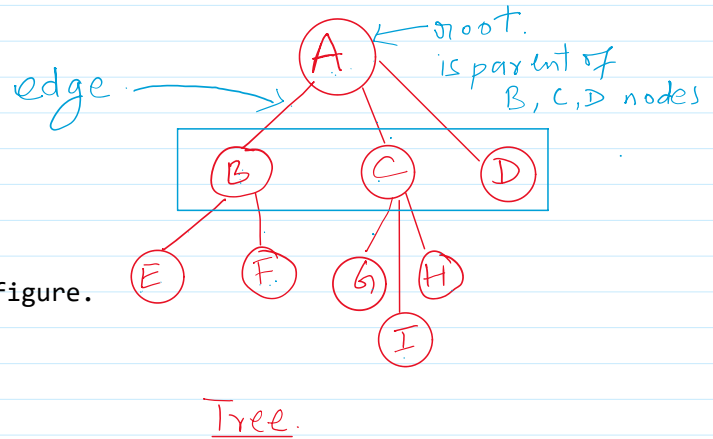
What is Tree in Computer Science?

A **Tree** is a non-linear data structure used to store hierarchical data.

Unlike arrays or linked list, tree represent data in a parent-child relationship.

Example:

- File systems
- Organizational hierarchy
- XML/HTML structure
- Database indexing



Root Node: The topmost node of a tree.

Example: A is the root node in the given figure.

Parent Node: A node having child nodes

Example: A is a parent of B, C, D.

Child Node: Nodes connected below a parent node.

Example: B, C, D are child nodes of A.

Leaf node: Nodes having no children

Example: E, F, G, I, H, D in our diagram.

Siblings: Node having same parent.

Example: B, C, D are siblings having A as parent.

Edge: Connection between two nodes

Example: A-----B

Height of Tree: Maximum number of edges from root to deepest node.

Example: Height = 2 meaning Maximum number of edges from root to deepest node in our diagram.

Types of Trees

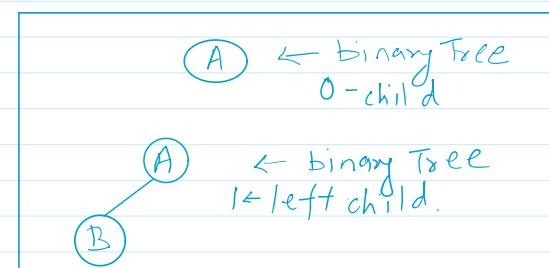
- General Tree
- Binary Tree
- Binary Search Tree
- AVL tree
- Heap
- B-Tree
- Expression Tree
- Recursion Tree
- Red-Black Tree
- And many more.

Binary Tree

A **Binary Tree** is a tree in which each node can have at most:

- One left child
- One Right child
- Or No-child

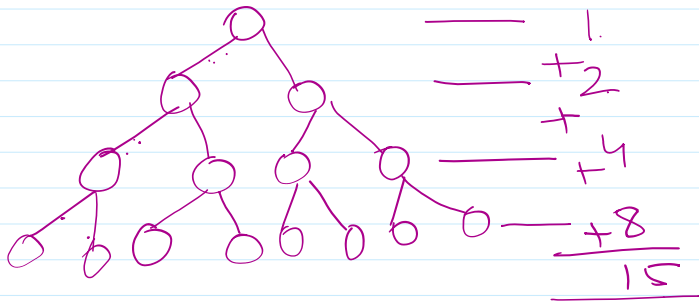
Properties of Binary Tree.



Properties of Binary Tree.

⊗ Maximum nodes at level $L = 2^L$

⊗ Maximum ^{number of} nodes in tree of height h .



$$= 2^{h+1} - 1 \quad [\because h = \text{height}]$$

$$= 2^{3+1} - 1 = 16 - 1 = \underline{15}$$

⊗ Minimum height of binary tree with $n = 15$ node.

$$2^{h+1} - 1 = 15 \quad \text{find } h.$$

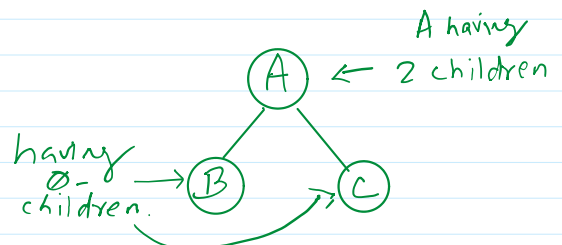
$$\begin{aligned} 2^{h+1} - 1 &= 15 \\ 2^{h+1} &= 16 \\ 2^{h+1} &= 2^4 \\ h+1 &= 4 \\ h &= 4 - 1 = 3 \end{aligned}$$

$$h_{\min} = \log_2(n+1) - 1$$

Types of Binary Trees

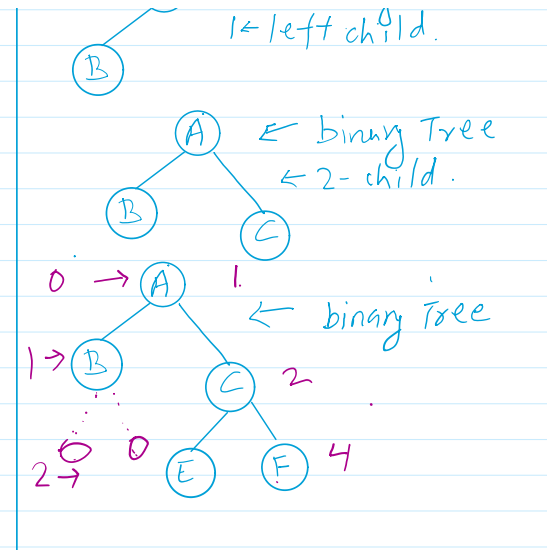
1. Full Binary Tree: Every node has either:

- ♦ 0-children
- ♦ Or 2-children

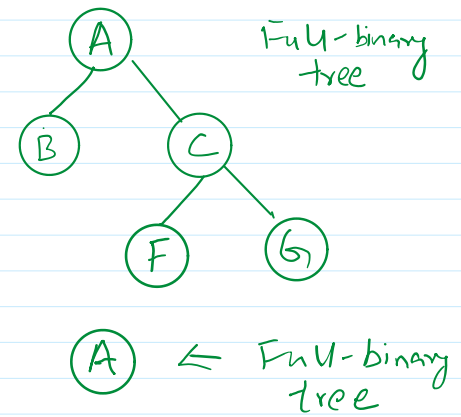
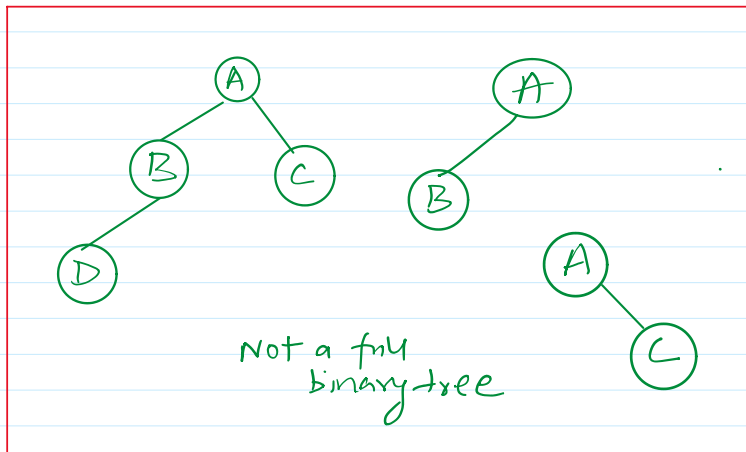


(A)

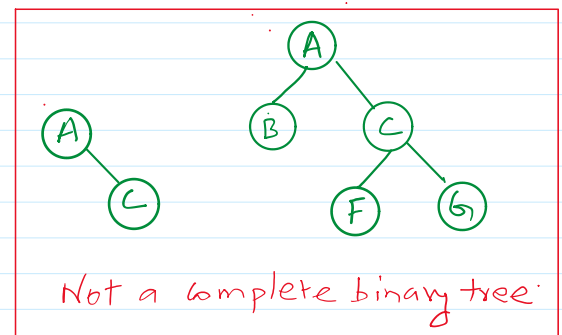
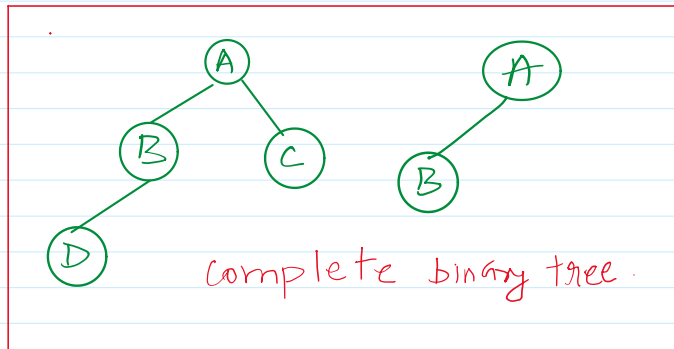
Full binary



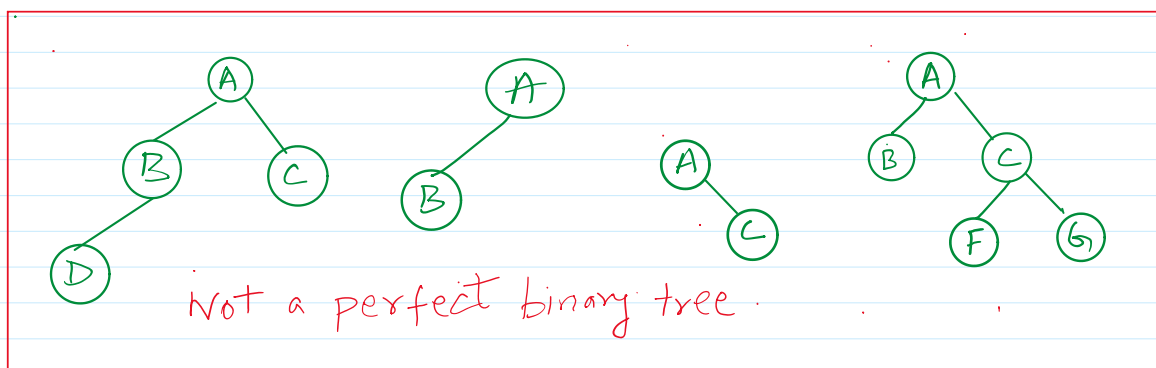
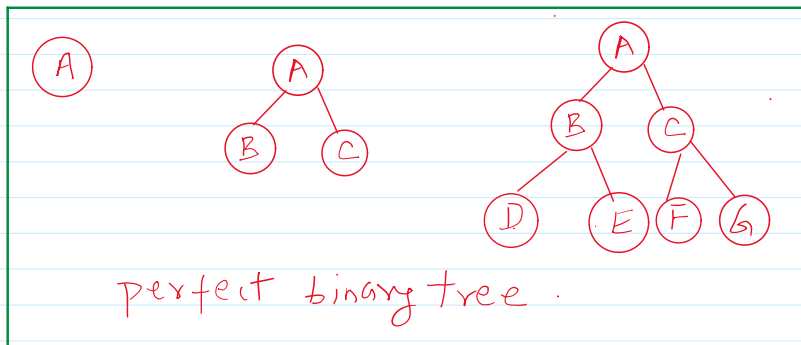
Structure of Binary Tree.



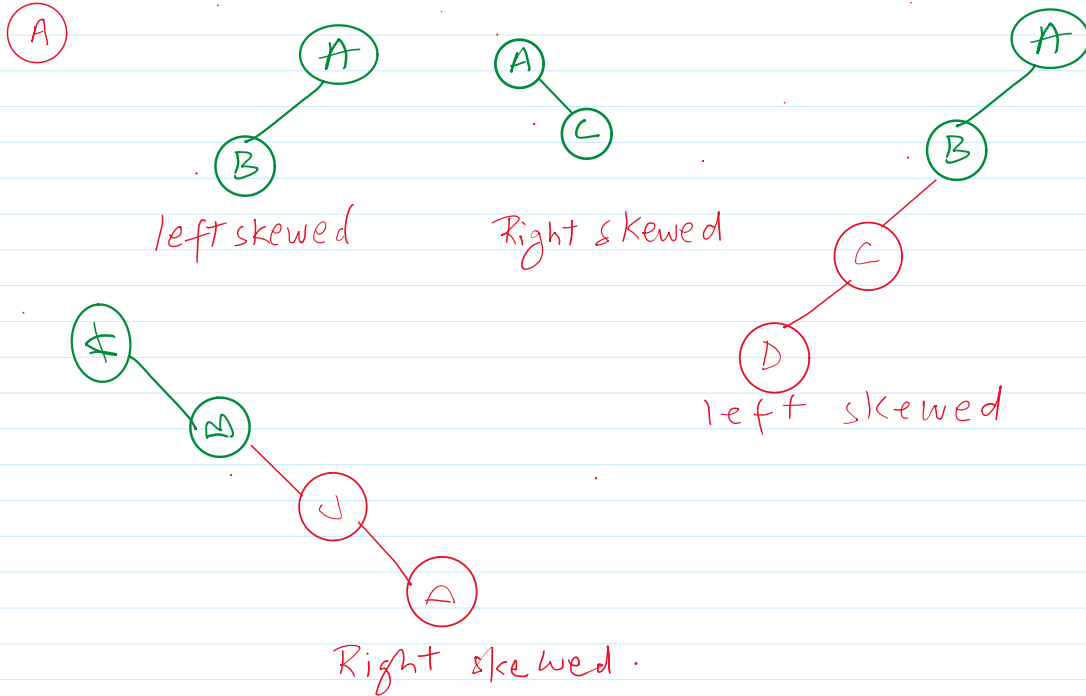
2. **Complete Binary Tree:** All levels completely filled except possibly last level. Last level filled from left to right.



3. **Perfect Binary Tree:** All internal nodes have 2 children and all leaf nodes are at same level.



4. Degenerated Binary Tree: Each parent has only one child, it looks like linked list.



```
class Node {
```

```
    int data;  
    Node left, right;
```

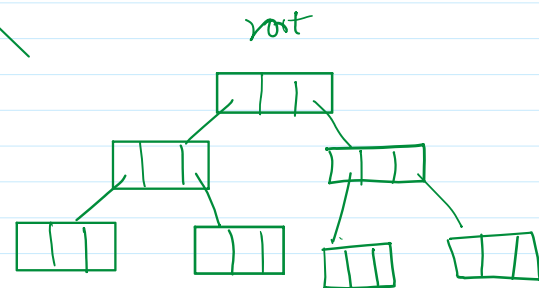
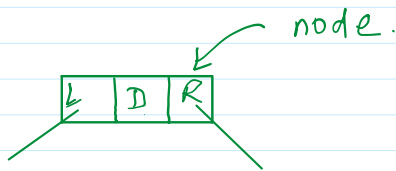
```
    Node() {}
```

```
    Node(data) {}
```

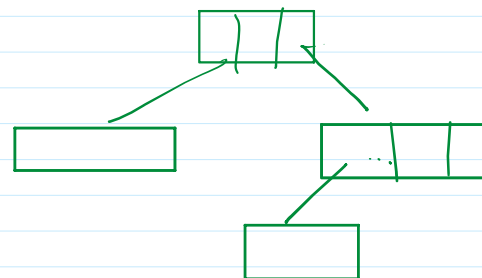
```
    // set and get methods
```

```
}
```

```
class BinaryTree {
```



Doubly Linked list logically hierarchical is called binary tree structure.



```
package BinaryTree;  
public class Node {
```

```

private int data;
private Node left, right;
public Node(){
    data = 0;
    left = right = null;
}
public Node(int data) {
    this.data = data;
}
public int getData() {
    return data;
}
public void setData(int data) {
    this.data = data;
}
public Node getLeft() {
    return left;
}
public void setLeft(Node left) {
    this.left = left;
}
public Node getRight() {
    return right;
}
public void setRight(Node right) {
    this.right = right;
}
@Override
public String toString() {
    return "Node [data=" + data + "]";
}
}

```

```

package BinaryTree;
import java.util.Scanner;
public class BinaryTreeDemo {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        Node root = new Node(100);
        Node left = new Node();
        System.out.println("Enter data for node: ");
        left.setData(sc.nextInt());
        root.setLeft(left);
        Node right = new Node();
        System.out.println("Enter data for node: ");
        right.setData(sc.nextInt());
        root.setRight(right);

        // System.out.println("Root node = " + root);
        // System.out.println("Left child = "+left);
        // System.out.println("Right child = " + right);

        System.out.println("Root node = " + root);
        System.out.println("Left child = "+root.getLeft());
        System.out.println("Right child = " + root.getRight());
        Node rightLeft = new Node(150);
        right.setLeft(rightLeft);
        System.out.println("Adding a node on the left side of right child:");
        System.out.println("Root node = " + root);
    }
}

```

```
        System.out.println("Left child = "+root.getLeft());  
        System.out.println("Right child = " + root.getRight());  
        System.out.println("Left of Right Child = " + root.getRight().getLeft());  
    }  
}
```

Output:

Enter data for node:

50

Enter data for node:

200

Root node = Node [data=100]

Left child = Node [data=50]

Right child = Node [data=200]

Adding a node on the left side of right child:

Root node = Node [data=100]

Left child = Node [data=50]

Right child = Node [data=200]

Left of Right Child = Node [data=150]